Traducción Automática Neural Inglés–Español Mediante una Arquitectura Seq2Seq Clásica Basada en LSTM

Carlos Valdez P.
Maestría en Inteligencia Artificial
Universidad San Francisco de Quito
Quito, Ecuador
cwvaldezp@estud.usfq.edu.ec

***Resumen*—Este trabajo presenta el desarrollo y evaluación de un sistema de traducción automática neural del inglés al español basado en una arquitectura clásica Sequence-to-Sequence (Seq2Seq) con redes Long Short-Term Memory (LSTM). El conjunto de datos utilizado corresponde a un corpus paralelo almacenado en el archivo `spa.txt`, con un total de 118964 pares de oraciones. Tras un proceso de limpieza y normalización textual, se conservaron 118650 pares válidos, los cuales fueron divididos en subconjuntos de entrenamiento, validación y prueba. A partir de estos datos se construyeron vocabularios independientes para ambos idiomas y se implementó un modelo encoder-decoder con embeddings de 128 dimensiones, estado oculto de 256 unidades y regularización por dropout. El entrenamiento base alcanzó una mejor pérdida de validación de 3.1956 en la época 12, mientras que el proceso de fine-tuning redujo la pérdida en el conjunto de prueba de 3.2492 a 3.1942. Adicionalmente, el modelo obtuvo un puntaje BLEU de 0.1226, consistente con un sistema Seq2Seq básico sin mecanismo de atención. Los resultados evidencian que la arquitectura logra capturar relaciones semánticas elementales, aunque presenta limitaciones asociadas al cuello de botella de información. Finalmente, se discute la incorporación de atención como línea natural de mejora futura.**

***Index Terms*—Traducción automática neural, Seq2Seq, LSTM, encoder-decoder, aprendizaje profundo, BLEU.**

I. Introducción

La traducción automática neural se ha consolidado como una de las aplicaciones más relevantes del aprendizaje profundo dentro del procesamiento de lenguaje natural. A diferencia de los enfoques estadísticos tradicionales, los modelos neuronales permiten aprender representaciones distribuidas de las secuencias y modelar dependencias complejas entre palabras y estructuras sintácticas.

Entre las arquitecturas más representativas se encuentra el enfoque Sequence-to-Sequence, en el cual una red *encoder* transforma la oración de entrada en una representación latente, mientras que una red *decoder* utiliza dicha representación para generar la traducción token por token. Aunque este enfoque constituyó un gran avance en traducción automática, sus variantes clásicas presentan limitaciones importantes cuando toda la información de la secuencia fuente debe comprimirse en un único vector de contexto.

En este trabajo se implementó un sistema de traducción inglés–español basado en una arquitectura Seq2Seq clásica con LSTM, sin mecanismo de atención. El objetivo fue construir una línea base funcional, analizar su comportamiento durante el entrenamiento y evaluar sus capacidades de traducción mediante métricas cuantitativas y ejemplos cualitativos. Además, se realizó una etapa de fine-tuning para examinar si era posible mejorar la generalización del modelo a partir del mejor checkpoint obtenido durante el entrenamiento base.

II. Trabajos Relacionados

El enfoque Sequence-to-Sequence (Seq2Seq) fue introducido por Sutskever et al. [1] como una arquitectura capaz de modelar transformaciones entre secuencias de longitud variable mediante redes recurrentes profundas. Posteriormente, Cho et al. [2] consolidaron el uso de esquemas encoder-decoder en tareas de traducción automática neuronal.

Sin embargo, el uso de un vector de contexto fijo evidenció limitaciones en la representación de oraciones largas. Para abordar este problema, Bahdanau et al. [3] propusieron el mecanismo de atención, permitiendo al decoder acceder dinámicamente a los estados del encoder durante la generación de la secuencia. Luong et al. [4] ampliaron estos enfoques, demostrando mejoras consistentes en desempeño.

Adicionalmente, se revisaron implementaciones prácticas y tutoriales orientados a traducción automática neuronal basados en PyTorch y otros recursos técnicos especializados [5]–[7].

III. Preparación del Conjunto de Datos

III-A. Carga del DataSet

Se utilizó el archivo `spa.txt` como dataset paralelo inglés–español. El archivo fue almacenado localmente con un tamaño de 8042772 bytes y contiene un total de 118964 líneas, cada una correspondiente a un par de traducción separado por tabulación.

Durante la inspección inicial se verificó la correcta codificación del archivo y la presencia de pares válidos de traducción.

III-B. Limpieza y normalización

Con el fin de homogenizar las secuencias y reducir variabilidad innecesaria, se aplicó una función de normalización textual a todos los pares. Este proceso incluyó conversión a

minúsculas, eliminación de espacios redundantes y separación explícita de signos de puntuación como puntos, comas, signos de interrogación y exclamación.

Cuadro I
EJEMPLOS DE NORMALIZACIÓN TEXTUAL

Texto original	**Texto normalizado**
Go.	go .
Who?	who ?
¡Corre!	¡ corre !
Váyase.	váyase .

Después del preprocesamiento se conservaron 118650 pares de oraciones válidos.

III-C. Partición de los datos

El dataset preprocesado fue dividido en tres subconjuntos utilizando una semilla fija de 42 para asegurar reproducibilidad. Se definió una partición de 70 % para entrenamiento y 30 % para separación temporal, tras lo cual esta última se dividió nuevamente para obtener validación y prueba.

Cuadro II
DISTRIBUCIÓN DEL CONJUNTO DE DATOS

Subconjunto	**Número de pares**
Entrenamiento	83055
Validación	23730
Prueba	11865
Total	118650

Esta división permitió entrenar el modelo con una cantidad suficiente de ejemplos y, al mismo tiempo, reservar conjuntos independientes para selección de checkpoints y evaluación final.

IV. METODOLOGÍA

IV-A. Construcción del vocabulario y DataLoaders

A partir de los pares preprocesados se construyeron vocabularios independientes para el idioma fuente y el idioma objetivo. El vocabulario inglés quedó conformado por 11686 tokens únicos, mientras que el vocabulario español alcanzó 21986 tokens únicos. Tras añadir tokens especiales para inicio de secuencia, fin de secuencia, padding y desconocidos, los tamaños finales fueron 11690 para inglés y 21990 para español.

Cada oración fue convertida a una secuencia de índices enteros. Para el encoder se utilizó el formato `<sos>` oración `<eos>`, mientras que para el decoder se definieron entradas y salidas desplazadas para permitir el entrenamiento autoregresivo.

Posteriormente se creó una clase `TranslationDataset` y se organizaron los datos mediante *DataLoaders* con tamaño de lote de 64. Esto produjo 1298 lotes para entrenamiento, 371 para validación y 186 para prueba.

Durante la inspección de un lote real se observó que las secuencias eran rellenadas con padding hasta igualar la longitud máxima del lote, junto con un vector de longitudes reales para aprovechar `pack_padded_sequence` en el encoder.

IV-B. Arquitectura Seq2Seq basada en LSTM

El sistema implementado corresponde a una arquitectura Seq2Seq clásica compuesta por un encoder LSTM y un decoder LSTM. El encoder recibe la secuencia fuente y devuelve los estados ocultos y de memoria finales, los cuales inicializan el decoder. Este último genera la traducción palabra por palabra. La Fig. 1 muestra la estructura general del modelo propuesto.

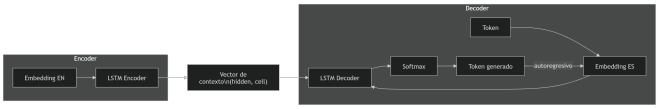

Figura 1. Arquitectura Seq2Seq clásica basada en LSTM utilizada para traducción automática neural.

Formalmente, dada una secuencia fuente $x = (x_1, x_2, ..., x_T)$, el encoder produce una representación latente a través de:

$$h_t = \text{LSTM}(x_t, h_{t-1}) \tag{1}$$

El estado final del encoder se utiliza como contexto inicial del decoder, que modela la probabilidad condicional de la secuencia objetivo:

$$s_t = \text{LSTM}(y_{t-1}, s_{t-1}, c) \tag{2}$$

y la distribución del siguiente token se obtiene mediante:

$$P(y_t|y_{<t}, c) = \text{Softmax}(W s_t + b) \tag{3}$$

El modelo final contiene 10752998 parámetros entrenables.

Cuadro III
HIPERPARÁMETROS PRINCIPALES DEL MODELO

Parámetro	**Valor**
Vocabulario fuente	11690
Vocabulario objetivo	21990
Embedding dim	128
Hidden dim	256
Capas LSTM	1
Dropout	0.2
Batch size	64
Teacher forcing	0.3
Learning rate inicial	0.001
Gradient clipping	1.0

IV-C. Entrenamiento del modelo

El entrenamiento se realizó con el optimizador Adam y una tasa de aprendizaje inicial de 0.001. Como función de pérdida se utilizó entropía cruzada con enmascaramiento del token de padding mediante `ignore_index`. Además, se

aplicó *gradient clipping* con valor 1.0 para reducir el riesgo de gradientes explosivos.

El modelo fue entrenado por un máximo de 50 épocas sobre GPU (`cuda`), usando una política de *early stopping* con paciencia de 10 épocas sin mejora en validación. Durante el proceso se almacenaron checkpoints del modelo con sus respectivas pérdidas de validación.

La mejor pérdida de validación durante el entrenamiento base fue 3.1956 en la época 12, tras lo cual el proceso continuó hasta la época 22, donde se activó el criterio de early stopping.

IV-D. Fine-tuning

Una vez concluido el entrenamiento base, se cargó el mejor checkpoint y se ejecutó una fase adicional de fine-tuning. En esta etapa se utilizó nuevamente Adam, pero con una tasa de aprendizaje reducida a 0.0003. Se definió un máximo de 10 épocas y una paciencia de 3 épocas sin mejora.

El objetivo del fine-tuning fue refinar los parámetros del modelo a partir del mejor estado encontrado en entrenamiento base y observar si ello producía mejoras en el conjunto de prueba.

V. Resultados y Análisis

V-A. Evolución de pérdidas durante el entrenamiento

Durante el entrenamiento, la pérdida en el conjunto de entrenamiento disminuyó de forma sostenida desde 5.141 en la época 1 hasta 1.605 en la época 22. Por otro lado, la pérdida de validación mostró una caída rápida en las primeras épocas, alcanzando su mejor valor en la época 12 con 3.1956, tras lo cual tendió a estabilizarse y presentar un ligero incremento.

Este comportamiento indica que el modelo logró aprender patrones relevantes de traducción, aunque también evidencia una brecha progresiva entre entrenamiento y validación, lo cual sugiere la presencia de sobreajuste moderado. La Fig. 2 muestra la evolución de ambas pérdidas durante el proceso de optimización.

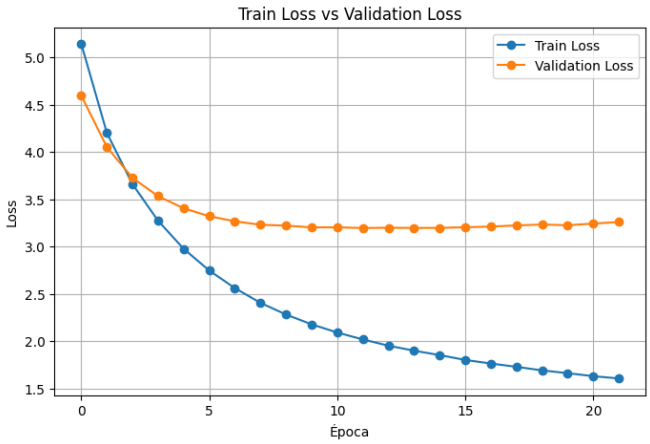

Figura 2. Evolución de la pérdida de entrenamiento y validación durante el entrenamiento del modelo Seq2Seq.

V-B. Evaluación de checkpoints

Se evaluaron los checkpoints generados durante el entrenamiento con el fin de analizar su comportamiento en validación y prueba. El mejor desempeño se obtuvo en la época 12, con una pérdida de validación de 3.1956 y una pérdida de prueba de 3.2492.

Los resultados muestran que los checkpoints cercanos al mínimo de validación también presentan desempeños similares en prueba, lo que sugiere una zona de convergencia relativamente estable del modelo.

Cuadro IV
Mejores checkpoints evaluados

Época	Val Loss	Test Loss
12	**3.1956**	**3.2492**
14	3.1967	3.2523
11	3.2033	3.2541
10	3.2025	3.2552
15	3.1970	3.2561
13	3.1984	3.2595

V-C. Resultados del fine-tuning

El proceso de fine-tuning permitió mejorar el desempeño del modelo en el conjunto de prueba. Mientras el modelo base alcanzó una pérdida de 3.2492, el modelo ajustado redujo este valor a 3.1942.

Este resultado indica que continuar el entrenamiento desde el mejor checkpoint, utilizando una tasa de aprendizaje menor, permite refinar las representaciones aprendidas y mejorar ligeramente la capacidad de generalización.

La Fig. 3 muestra la evolución de la pérdida durante la fase de fine-tuning, donde se observa una reducción adicional respecto al modelo base.

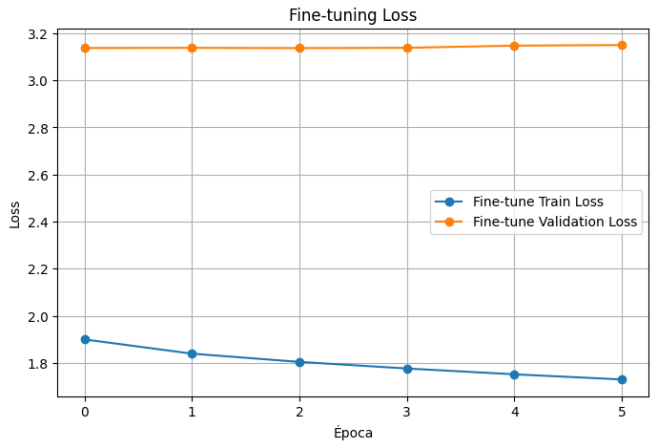

Figura 3. Evolución de la pérdida durante el proceso de fine-tuning del modelo Seq2Seq.

Durante el proceso de fine-tuning se observa una reducción adicional en la pérdida de validación respecto al modelo base, lo cual indica una mejora moderada en la capacidad de generalización. Este comportamiento sugiere que continuar el

entrenamiento desde el mejor checkpoint permite refinar las representaciones internas aprendidas por el modelo Fig. 4 .

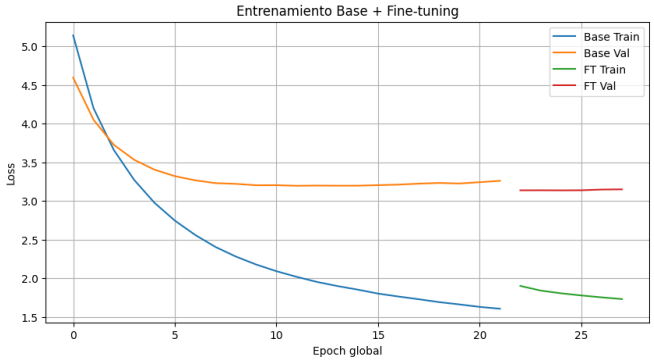

Figura 4. Evolución global de la pérdida durante el entrenamiento base y el proceso de fine-tuning del modelo Seq2Seq.

Se observa que, tras la fase inicial de entrenamiento, el proceso de fine-tuning permite una reducción adicional en la pérdida de validación, evidenciando una mejora en la capacidad de generalización del modelo.

V-D. Evaluación cualitativa

En la evaluación manual, el modelo generó traducciones con coherencia semántica parcial, aunque también presentó errores de repetición, omisión y concordancia gramatical. En frases simples como “that’s right .” el sistema produjo “es correcto .”, lo cual constituye una traducción aceptable.

Sin embargo, en ejemplos más complejos se observaron dificultades típicas de arquitecturas Seq2Seq sin atención, como repeticiones (“hombre alto alto”), selecciones léxicas incorrectas (“el luz .”), y estructuras incompletas (“noticia lo hizo hizo .”). Estos resultados evidencian limitaciones en el modelado de dependencias largas y en el control de la generación autoregresiva.

Cuadro V
EJEMPLOS CUALITATIVOS DE TRADUCCIÓN

Entrada (EN)	**Referencia (ES)**	**Predicción**
that’s right .	eso es .	es correcto .
step aside .	hazte a un lado .	fue paso .
the news made him happy .	la noticia le hizo feliz .	noticia lo hizo hizo .
i have not seen him in months .	hace meses que no lo veo .	le he visto en meses .
tom is one of my closest friends .	tom es uno de mis amigos más íntimos .	es uno de mis amigos mis amigos

V-E. Métrica BLEU

Para complementar la evaluación por pérdida, se calculó el puntaje BLEU sobre el conjunto de prueba. El modelo obtuvo un valor de 0.1226.

Este resultado ubica al sistema dentro de un rango de desempeño básico, lo cual es coherente con una arquitectura Seq2Seq clásica sin atención.

$$BLEU = 0.1226 \tag{4}$$

VI. DISCUSIÓN

Los resultados cuantitativos y cualitativos muestran que el modelo implementado es capaz de aprender relaciones de traducción entre el inglés y el español, especialmente en estructuras relativamente simples o frecuentes en el corpus. La reducción sostenida de la pérdida de entrenamiento y la mejora adicional obtenida mediante fine-tuning confirman que el sistema logró capturar patrones semánticos útiles.

No obstante, el comportamiento del modelo también deja en evidencia las limitaciones propias de una arquitectura encoder-decoder clásica sin atención. El decoder depende únicamente del estado final del encoder, lo que obliga a comprimir toda la información de la oración fuente en una representación fija. Este cuello de botella se vuelve especialmente problemático para oraciones largas o con estructura compleja.

Los errores observados en repetición de palabras, omisión de elementos y pérdida de concordancia son consistentes con este tipo de arquitectura. Del mismo modo, el valor BLEU de 0.1226 sugiere que, aunque el sistema constituye una línea base funcional, todavía está lejos de ofrecer traducciones robustas o de alta calidad.

A partir de la revisión posterior de fuentes técnicas y tutoriales especializados, se concluye que una línea natural de mejora consiste en incorporar un mecanismo de atención, el cual permitiría al decoder consultar dinámicamente los estados ocultos del encoder durante cada paso de generación. Esto debería reducir la pérdida de información y mejorar tanto la calidad gramatical como la fidelidad semántica de las traducciones.

VII. CONCLUSIONES

En este trabajo se implementó desde cero un sistema de traducción automática neural inglés–español basado en una arquitectura Seq2Seq clásica con LSTM. El proceso incluyó carga y limpieza del corpus, construcción de vocabularios, definición de DataLoaders, implementación del encoder y decoder, entrenamiento base, selección de checkpoints, fine-tuning e inferencia final.

A partir de un corpus de 118964 líneas se conservaron 118650 pares válidos, distribuidos en 83055 ejemplos de entrenamiento, 23730 de validación y 11865 de prueba. El modelo entrenado, con 10752998 parámetros, alcanzó una mejor pérdida de validación de 3.1956 en la época 12. Posteriormente, el fine-tuning mejoró la pérdida en prueba de 3.2492 a 3.1942. Además, el valor BLEU obtenido fue 0.1226.

En conjunto, estos resultados muestran que el modelo es funcional como línea base y que logra producir traducciones semánticamente razonables en ciertos casos, aunque todavía presenta errores recurrentes de repetición, simplificación y pérdida estructural. Como trabajo futuro, se propone incorporar mecanismos de atención, explorar arquitecturas bidireccionales y comparar el desempeño frente a enfoques basados en Transformers.

REFERENCIAS

[1] I. Sutskever, O. Vinyals y Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems*, 2014.

[2] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014.

[3] D. Bahdanau, K. Cho y Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *International Conference on Learning Representations*, 2015.

[4] M. Luong, H. Pham y C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 2015.

[5] H. Jain, "Machine translation seq2seq LSTMs," Kaggle, [Online]. Available: https://www.kaggle.com/code/harshjain123/machine-translation-seq2seq-lstms

[6] PyTorch, "Sequence to sequence translation tutorial," [Online]. Available: https://docs.pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html

[7] Google, "Neural machine translation (seq2seq)," [Online]. Available: https://google.github.io/seq2seq/nmt/